

CS 370 – Project 5A, Threads

Purpose: Become familiar with operating system interaction, threading, and race conditions.

Points: 100 40 for program and 60 for write-up

Scoring will include functionality, documentation, coding style, and write-up

Assignment

In recreational mathematics, a Duck number is a positive integer where one, and only one of the digits is a zero (excluding any leading zero's). For example, the numbers 1234, 100012, and 120230 are not Duck numbers. The numbers 12012, 10146, and 1230 are Duck numbers.

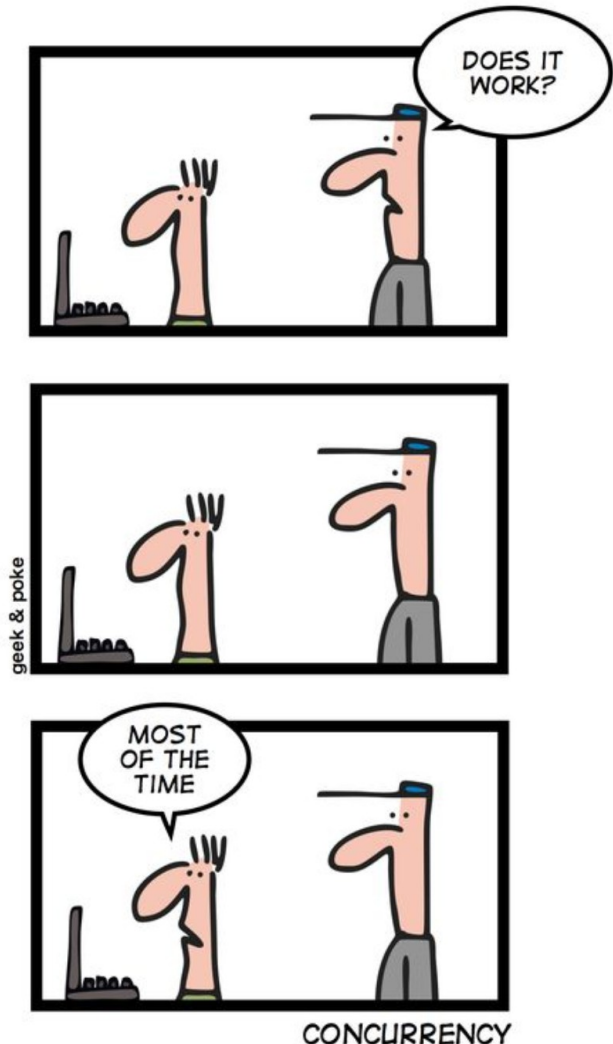
Write a C-language program to find the count of Duck numbers between 1 and some user provided limit. In order to improve performance, the program should use threads to perform some of the computations in parallel. The program should read the thread count option and number limit from the command line in the following format; `“./duckNums -th <number> -lm <number>”`. For example, for 4 threads and a limit of 1,000,000,000:

```
./duckNums -th 4 -lm 1000000000
```

The following routines, called by the a main, should be created:

- A boolean function, ***getParams()***, that reads and verifies the command line arguments. This includes error and range checking based on provided parameters. The error strings are defined in the example output. Due to the potential size of the limit, unsigned long data type should be used.
- A void function, ***duckCntThread()***, that will be called as a thread function and determine the count of Duck numbers. The thread must perform updates to the global variables in a critical section and avoid race conditions. See below sections for additional detail.

Implement this specific threaded problem in C (not C++). The program must compile and execute under Ubuntu (and will only be tested with Ubuntu). Assignments not executing under Ubuntu will not be scored. See additional guidance on following pages.



Thread Function

The general flow of the *duckCntThread()* thread function would be as follows:

- Obtain the next block of numbers to check (via global variable, *currentIndex*)
 - increment the global counter by the block size
- While the next current number in the block is $\leq$ *userLimit* (globally available);
 - Check if the number is a Duck number
 - If the number is a Duck number, increment a local Duck number count
- After all numbers in the block have been checked, update the global duck number count.

Incrementing, obtaining, and checking the next block of numbers, *currentIndex* variable, must be performed within a critical section (i.e., locked and unlocked).

Once the count of Duck numbers for the block has been found, the global duck number counter should be updated. The update for the *duckNumberCount* must also use a lock.

It is recommended to complete and test the program initially with only the single sequential thread before testing the parallel threads. In order to debug, you may wish to temporarily insert a direct call (non-threaded) to the *duckCntThread()* function (temporarily for testing and debugging).

Global Parameters

In order to use threading functions and semaphores in C, you will need the below include files:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>
#include <time.h>
#include <stdbool.h>
#include <errno.h>                // for errno
```

The following should be declared globally and used as parameters for the program.

```
const unsigned long  MIN_THREADS = 1;
const unsigned long  MAX_THREADS = 30;
const unsigned long  MIN_LIMIT = 100;
const unsigned long  BLOCK = 5000;
unsigned long         currCheck = 1;
unsigned long         userLimit = 0;
unsigned long         duckNumberCount = 0;
```

Project 5A - Timing Script

In order to obtain the times for the write-up a timing script is provided. After you download the script file, **p5atimer**, set the execute permission as follows:

```
ed-vm$ chmod +x p5atimer
ed-vm$ ./p5atimer duckNums
```

The script file will expect the name of the executable as an argument (as shown). The script may take a while to execute (i.e., 10+ minutes), but no interaction is required. The script will create an output file, **p5atimes.txt** and **p5aresults.txt** which will be included in the submission and the data be used create the write-up.

Compilation Options

Use the following compiler options

```
gcc -Wall -pedantic -pthread -o duckNums duckNums.c
```

Points will be removed for poor coding practices and unresolved warning messages.

Results Write-Up

When the program is working, complete additional timing and testing actions as follows;

- Use the provided script file to execute and time the working program.
 - The script will execute the program with 1, 2, 3, 4, 5, and 6 threads and create
 - the final results (in **p5aresults.txt**)
 - the execution times (in **p5atimes.txt**)
- Compute the speed-up¹ factor from the base sequential execution and the parallel execution times. Use the following formula to calculate the speed-up factor:

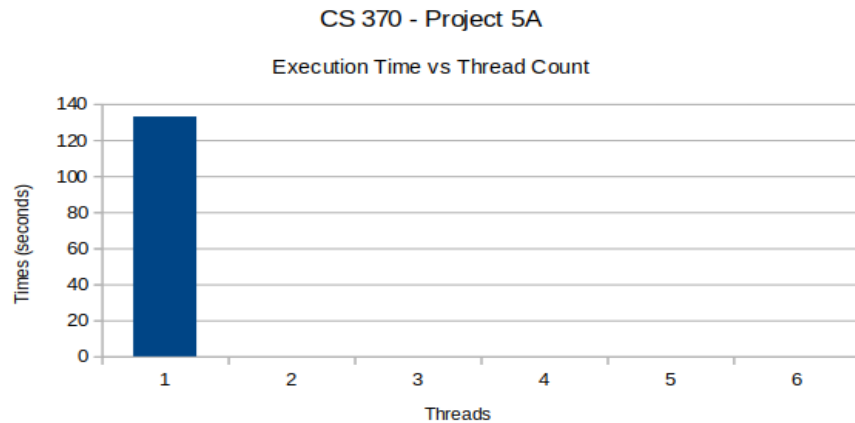
$$SpeedUp = \frac{ExecTime_{sequential}}{ExecTime_{parallel}}$$

Note, you may use the provided spreadsheet is assist in the calculations and creation of a chart.

- Create a copy of original working program that comments out the locks (both places). Execute the modified copy of the program using 6 threads five (5) times using a limit of 2,000,000,000. Include the results (final values only) for each test in your write up along with an explanation.
- Create another copy of original working program that eliminates the local block number count and just increments the global duck number count every time a duck number is found. Be sure to includes the appropriate locks. Use the timing script on this modified version of the program. Be sure to save the original results as the files will be over-written.

¹ For more information, refer to: <https://en.wikipedia.org/wiki/Speedup>

- Create a final write-up as follows:
 - The writeup must include the following sections:
 - Summary
 - Summary of machine the program was tested on and the final results for the 1-6 thread executions, including final results (from **p5aresults.txt**) and the execution times (from **p5atimes.txt**). This should include a chart (see example).
 - Speed-Up
 - Explain the speed changes between 1 through 6 threads and why the speed-up factor



changes as it does. Suggest some ways to improve.

- Lock Removal
 - Explain what happened and why when the locking calls were removed.
- Duck Counter Update Relocation
 - Explain what happened and why when the duck number count was moved as specified.
- The explanation part of the write-up (not including the output and timing data) should be less than 500 words. Overly long explanations will be not be scored. The report should include appropriate titles and section headers. Additionally, spelling and grammar will be part of the final report score.

Example Execution

The following are some example executions:

```
ed-vm%
ed-vm% ./duckNums -th 4 -lm
Error, invalid command line options.
ed-vm%
ed-vm% ./duckNums -tx 2 -lm 100000000
Error, invalid thread count specifier.
ed-vm%
ed-vm% ./duckNums -th 51 -lm 100000000
Error, thread count out of range.
ed-vm%
ed-vm% ./duckNums -th x -lm 100000000
Error, invalid thread count value.
ed-vm%
ed-vm% ./duckNums -th 2 -lmm 100000000
Error, invalid limit specifier.
ed-vm%
ed-vm% ./duckNums -th 3 -lm 10000x000
Error, invalid limit value.
ed-vm%
ed-vm% ./duckNums -th 2 -lm 1
Error, limit must be > 100.
ed-vm%
ed-vm% ./duckNums
Usage: ./duckNums -th <threadCount> -lm <limitValue>
ed-vm%
ed-vm% ./duckNums -th 2 -lm 100000000
CS 370 - Project #2
Duck Numbers Program

Hardware Cores: 12
Thread Count:   2
Numbers Limit:  100000000

Please wait. Running...

Duck Numbers Results
Count of Duck numbers from 1 to 100000000 is 36993276
ed-vm%
ed-vm%
```

Submission

When complete, submit:

- A copy of the final working C source code (not C++) file.
 - Do not submit the modified test versions (used for answering the write-up questions).
- A PDF of the write-up (as described above).

Submissions received after the due date/time will not be accepted.